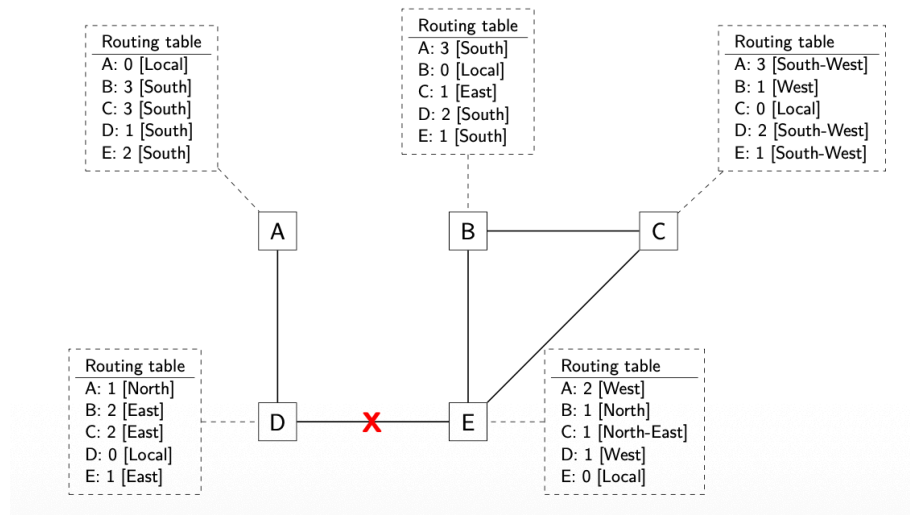
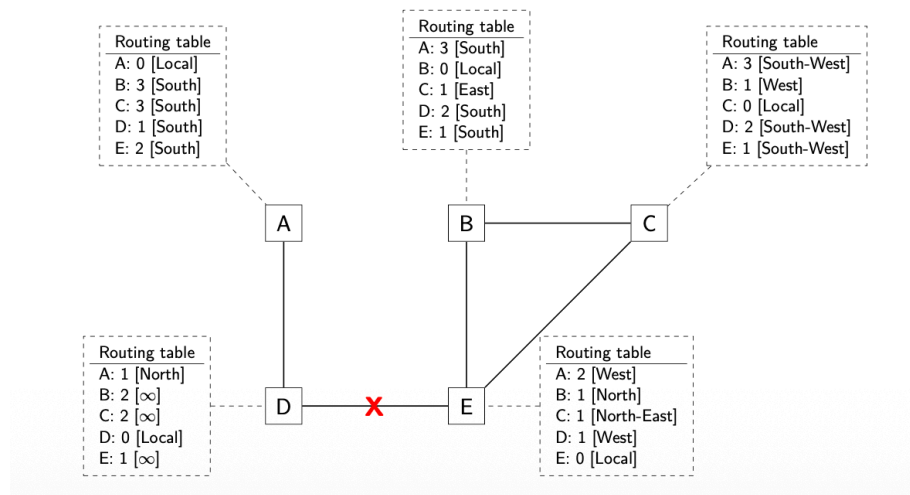


21 - 10

count to infinity



- D è il primo ad accorgersi della rottura e setta il costo a B C e E a ∞



- D ora è in stato in cui può ricevere un aggiornamento, ogni aggiornamento con una nuova distanza da B,C,E sarà inserito nella sua tabella di routing

```
# V : received Vector
# I : link over which vector is received
# R[]: the current routing table
```

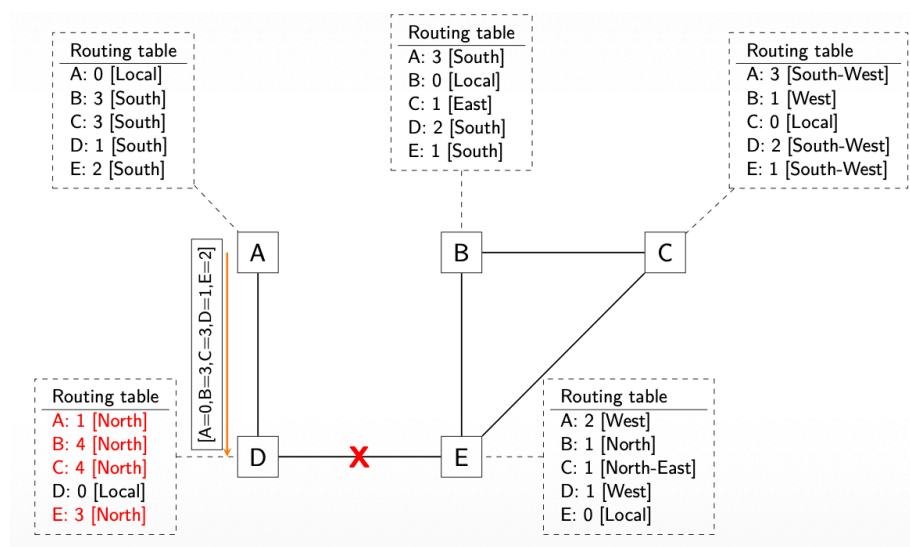
```
def received(V, I):
```

```

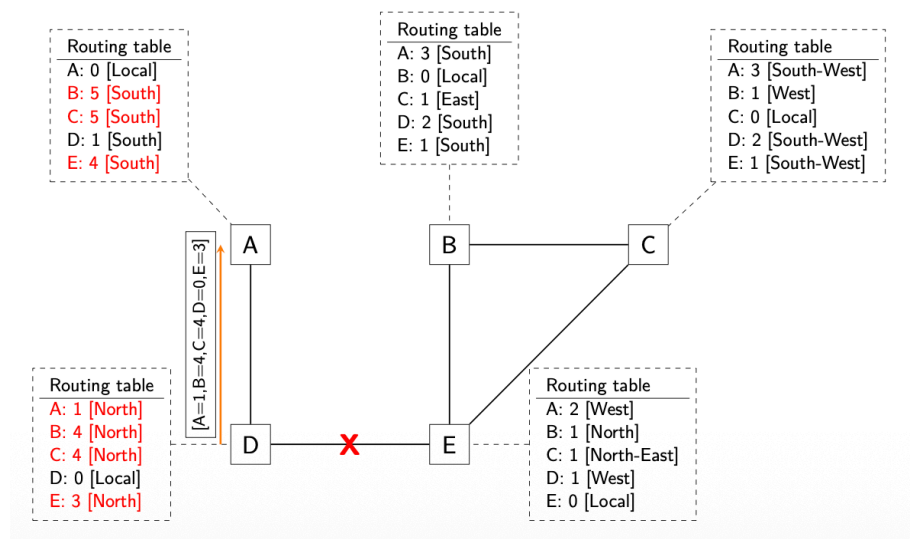
# received vector from link l
for d in V:
    if not (d in R):
        # new route
        R[d].link = l
        R[d].cost = V[d].cost + l.cost
        R[d].time = now()
    else:
        # existing route, should I update?
        if ((V[d].cost + l.cost) < R[d].cost) or (R[d].link == l):
            R[d].link = l
            R[d].cost = V[d].cost + l.cost
            R[d].time = now()

```

tuttavia A manda il DV prima di D



D aggiorna la sua routing table e conclude che può usare A come prossimo Hop per B, C ed E, dopo un po D manda il suo DV ad A



A aumenta la distanza a B, C, ed E e manda un altro DV, D riceve il DV e aggiorna la sua routing table aumentando la distanza. I risultato sono A e D che ad un certo punto continueranno a inviarsi DV e a incrementare la distanza fino a raggiungere overflow.

triggered updates

La correzione più semplice è inviare subito un DV quando una connessione fallisce: quando D imposta il costo a infinito, genera immediatamente un DV.



Può accadere che comunque D riceva un update prima di inviare il suo DV, o il DV mandato da D potrebbe venire scartato

cause del count to infinity

- deve esserci un loop topologico, una singola connessione full duplex già lo crea per cui accade sempre
- deve esserci un cambiamento nella topologia della rete non per forza una rottura della rete
- un pacchetto perso o il non utilizzo dei triggered updates

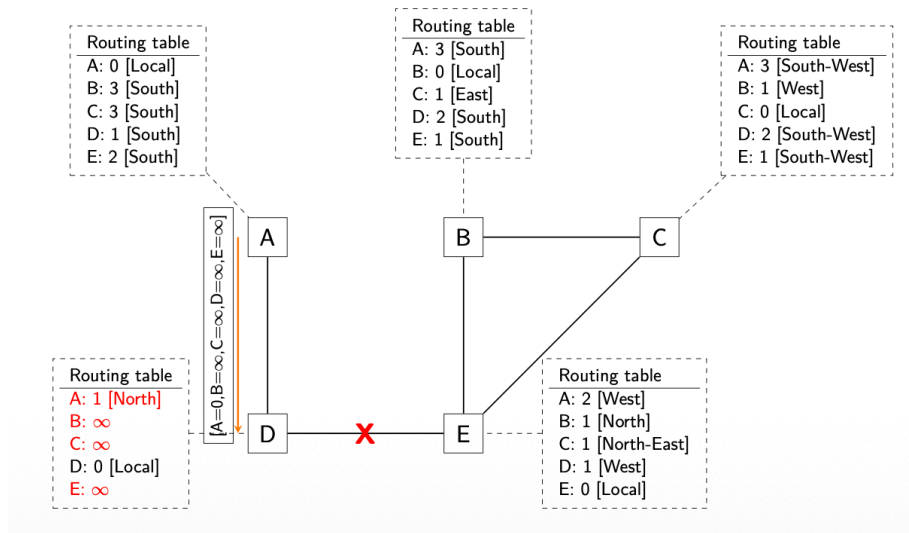
la causa alla radice è che il DV non contiene tutto il path usato per raggiungere la destinazione. Per ogni route è inclusa solo la distanza.

Split Horizon with Poison Reverse

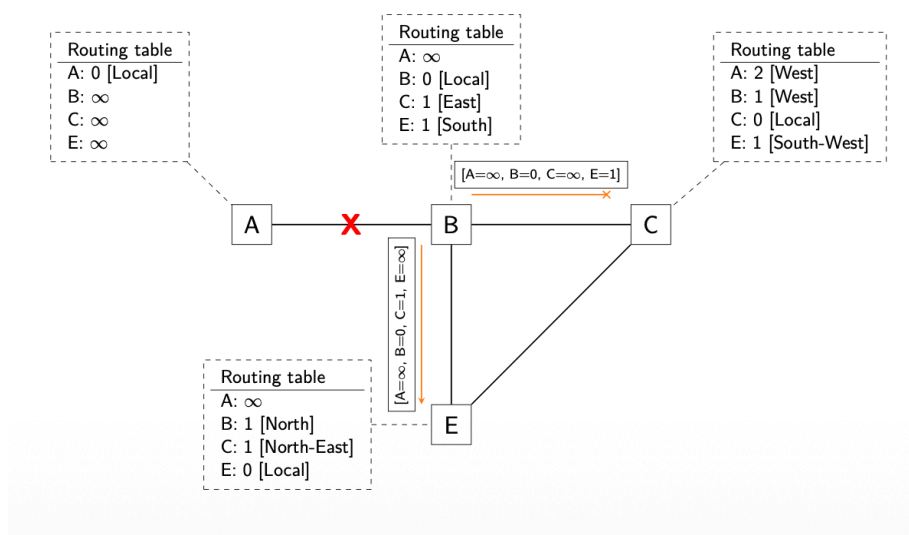
```
# Every N seconds:
for ifc in interfaces:
    # one vector for each interface
    v = Vector()
    for d in R:
        if R[d].link != ifc:
            v.add(Pair(d, R[d].cost))
        else:
            v.add(Pair(d, infinity))
```

Split Horizon perchè per un router l'orizzonte non è lo stesso, si divide tra i vicini usati come next hop e tutti gli altri.

poi il router fa poisoning sulle tabelle dei vicini, dicendo che non ha una rotta per le destinazioni la cui rotta passa per quel router stesso.



con il Poison Reverse D non aggiornerà la sua tabella di routing, non appena un DV arriverà ad A, aggiornerà correttamente la sua routing table.

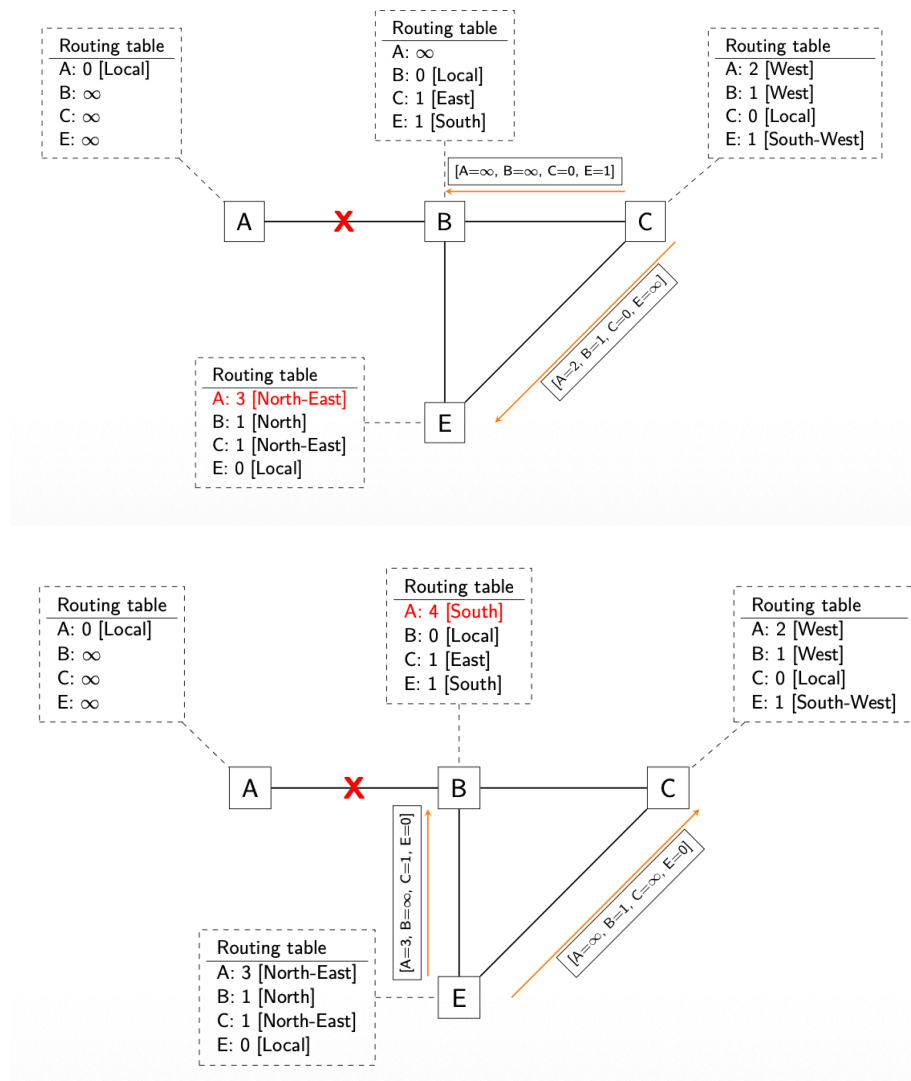


in questo caso A-B si rompe, B manderà il DV a C ed E, tutti i router usano poison reverse, ma E riceve correttamente il DV mentre C fallisce a riceverlo, ciò forma un black-hole (C-B).

Ora C manda il suo DV a B ed E, senza Poison reverse avremmo avuto un loop tra B e C.



E aggiornerà la sua tabella



! ed ecco un nuovo loop é generato tra E, B e C

per evitarlo è prevenire update di routes per un po' di tempo dopo la rottura, questo significa che se una route è a ∞ , il router non dovrebbe accettare un aggiornamento per un po di tempo.

Questo permette di evitare singoli eventi di packet loss e contare sul fatto che l'informaizione verra inviata ai vicini



L'unico modo per evitare il Count-to-infinity è che i routers mandino non solo la distanza ma anche tutto il path fino alla destinazione

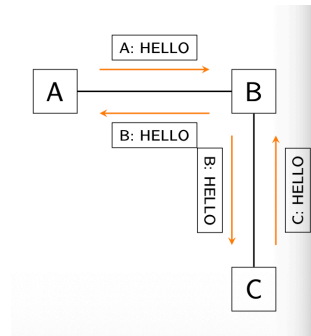
link state routing (LS)

Con questo modello i router si mandano messaggi per aiutarsi a capire la topologia della rete intera. la rete è rappresentata da un grafo orientato pesato (più il peso è alto peggiore la connessione).

il peso è proporzionale alla propagazione del delay sul link.

$$\frac{C}{\text{link} - \text{capacity}}$$

dove C è una costante, più alta la capacità del link, più basso il peso.



Ogni riuter ha un ID e manda un messaggio di **Hello** ogni N secondi a tutte le sue interfacce, **Hello** contiene l'adress del router, in questo modo un router scopre i suoi vicini.

Dopo di che ogni router crea un **link-state packet (LSP)** contenente:

```

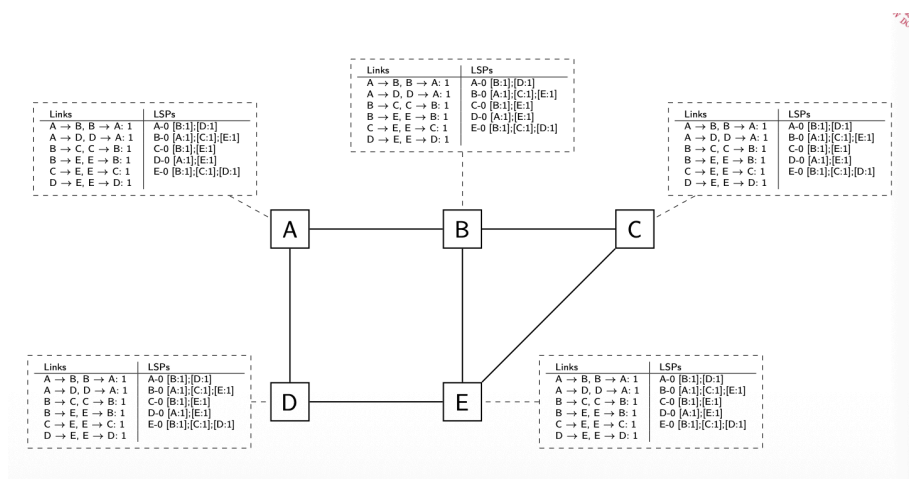
LSP.Router: identification //(adress) del mandante
LSP.seq: //sequenza dei numeri dell'LSP (aumenta ad ogni LSP)
LSP.Links[]: // link pubblicati con 2 campi
    LSP.links[i].Id : //identification del vicino
    LSP.links[i].cost: //peso del link
  
```

è usato un algoritmo di Flooding per distribuire il LSP a tutti i routers. Ogni router che implementa Flooding mantiene un Link State Database con i LSPD più recenti ricevuti dagli altri Router.

quando un router riceve un LSP prima verifica che ci sia già nel proprio Database, se non è presente manda LSP a tutti i vicini tranne a quello da cui l'ha ricevuto

```

# links is the set of all links on the router
# Router R LSP arrival on link l
if newer(LSP, LSDB(LSP.Router)) : # get last LSP from the DB, compare with current
    LSDB.add(LSP)                  # implicitly removes older LSP from same router
    for i in links:
        if i != l:
            send(LSP, i)
# else, LSP has already been flooded
  
```



link failure

quando un link fallisce i due router vicini si accorgono del danno grazie alla mancanza degli Hello negli ultimi $k * N$ secondi, i router che si rendono conto del danno generano un nuovo LSP che non contiene più la parte di rete danneggiata e lo diffondono.

nella fase transitoria di Flooding loop e black-holes possono essere generati, se un messaggio viene perso la convergenza potrebbe non venire raggiunta, nel maggiore dei casi LSP sono generati periodicamente.

- LS è più oneroso a livello di computazione del DV routing
- La rete internet ha centinaia di migliaia di nodi e usare un algoritmo tipo Dijkstra su un grafo simile richiede un potente hardware
- una rete grande comporta un'alta probabilità che qualcosa si rompa, LS protocol è molto più talkative, questo può indurre a overhead in termini di messaggi di controlli inviati

scalabilità LS

Internet ha milioni di routers e links, probabilmente si rompe un link al secondo, con LS vorrebbe dire che tutti i router dovrebbero ricalcolare Dijkstra ogni secondo, Internet routing non usa LS, LS è usato all'interno delle singole reti.



In ogni caso a livello network i pacchetti possono essere persi, corrotti o duplicati, I livelli superiori devono tenerlo in considerazione.